HXC-166 — dependency advisory triage (helix_agent)

HXC-166 — dependency advisory triage (helix_agent) — 2026-07-29

Field	Value
Revision	1
Created	2026-07-29
Last modified	2026-07-29
Status	active
Item	HXC-166 (Bug, Queued)
Scope	`submodules/helix_agent` @ `c740eaf1` — module `dev.helix.agent`
Trigger	204 open Dependabot advisories surfaced on publish
Pass type	**ANALYSIS ONLY — no dependency was upgraded, no `go.mod`/`go.sum` edited**

Table of contents

What this is and what it is not

This is a triage: a separation of the 204 advisories into those that plausibly reach code we deploy and those that do not, so that remediation effort can be spent where it changes our exposure.

It is not a clean bill of health. The component ships third-party code with known, publicly documented weaknesses. It still does after this pass. Triage changes what we know, not what we run.

Two distinct grounds are used to lower an advisory's priority, and they are **not** interchangeable. Every ranking decision below states which one it rests on:

- **Ground A — not deployed.** Evidence that the affected code is not built into, or copied into, any artifact we run. This is an evidence-backed claim about our deployment, verifiable from build files.
- **Ground B — not called.** Evidence from static call-graph analysis that our code never reaches the vulnerable symbol.

"Probably not exploitable" with neither Ground A nor Ground B behind it is a guess (§11.4.6) and is not used anywhere in this document. Where neither could be established, the advisory is listed as **undetermined**, not as low priority.

How the numbers were obtained

Advisory list — GitHub Dependabot API, authenticated via `gh` (token scopes `repo`, which satisfies the endpoint's `security_events` requirement):

```
gh api --paginate "repos/vasic-digital/HelixAgent/dependabot/
alerts?state=open&per_page=100"
```

Returned **exactly 204 open alerts**, matching the reported counts:

Severity	Count
critical	5
high	80
medium (reported as "moderate")	100
low	19
total	**204**

Note on remotes: the alerts live on **`vasic-digital/HelixAgent`**. The same query against `HelixDevelopment/HelixAgent` returns `[]` — Dependabot is enabled on one mirror only. Anyone checking the other remote sees a clean list; that is a reporting artifact, not a different risk posture.

Reachability — `govulncheck`, which was already installed at `/home/milos/go/bin/govulncheck` (no install, no root needed). This is the load-bearing tool for the Go side because it does **call-graph reachability analysis against our actual source**, not manifest version matching:

```
govulncheck ./...                    # symbol results
govulncheck -show verbose ./...      # plus package/module-level (unreachable) results
```

`govulncheck` covers Go only. **There is no equivalent reachability oracle applied to the 171 npm and 13 pip advisories** — see What could NOT be determined.

Deployment surface — build files, read directly: `docker-compose.protocols.yml`, per-directory `Dockerfile`s, `package.json` (workspaces/scripts), presence of `node_modules`, and the Go import graph.

Headline finding — the deployment-surface split

The 204 advisories are not distributed across our product. They are concentrated in in-tree copies of third-party projects that we do not build:

Tier	Location	Total	crit	high	med	low	Ground
T4	`mcp-servers/GitMCP/**` — vendored upstream, not built	139	4	43	76	16	A
T3	`sdk/web/` — our SDK, build toolchain	22	1	11	8	2	partial
T1	`go.mod` — the shipped Go binary	15	0	7	7	1	B (per-advisory)

Tier	Location	Total	crit	high	med	low	Ground
T4	`mcp-servers/postgres-mcp/ingest/` — not copied into image	13	0	12	1	0	A
T5	`Website/` — static site	6	0	3	3	0	A
T3	`pkg/api`, `plugins/packages/transport/go` — Go sub-modules	3	0	1	2	0	partial
T2	`plugins/mcp-server/` — **ours, containerised, in compose**	3	0	2	1	0	none
T5	`docs/research/go-elder-plinius-v3/**` — research artifact	2	0	1	1	0	A
T5	`ci/reporter/` — private CI helper	1	0	0	1	0	A

161 of 204 (78%) sit in T4/T5 — code that is in the tree but not in anything we deploy.

`mcp-servers/GitMCP` — 139 advisories, evidence for Ground A

This directory is a vendored copy of the upstream **`git-mcp`** project (gitmcp.io), not code we authored:

- `package.json` → `"name": "git-mcp"`, `"private": true`, `"license": "ISC"`, description *"GitMCP is a tool that allows you to get the documentation for a given repository."*
- It carries its own `README.md` (with upstream `github.com/user-attachments` image URLs), its own `SECURITY.md`, and its own `.github/workflows/`.
- It is a **Cloudflare Workers** + **React Router** app — `wrangler.jsonc`, `react-router.config.ts`, `vite.config.ts`, `tsconfig.cloudflare.json`. Its dependency set (`wrangler`, `hono`, `react-router`, `@remix-run/*`, `vite`, `rollup`, `turbo-stream`) is that of a hosted web app, not of an MCP process our agent spawns.

- It uses `pnpm-lock.yaml` while every other JS directory in the submodule uses `package-lock.json` — a foreign-origin marker.

Build wiring — the decisive evidence:

- There is **no** root `package.json` `workspaces` glob and **no** `Makefile` target referencing `mcp-servers`.
- `mcp-servers/GitMCP/node_modules` is **absent** — the dependency tree the 139 advisories describe has never been installed in this checkout.
- The single piece of build wiring is compose service `mcp-gitmcp` in `docker-compose.protocols.yml:381-384`:

```yaml
mcp-gitmcp:
  build:
    context: ./mcp-servers/GitMCP
    dockerfile: Dockerfile
```

`mcp-servers/GitMCP/Dockerfile` does not exist. Verified: `ls: cannot access 'mcp-servers/GitMCP/Dockerfile': No such file or directory`. The service cannot build; `docker compose build mcp-gitmcp` fails before any npm install happens.

Runtime wiring: the only reference from Go is `cmd/helixagent/main.go:4334 → cliagents.AgentGitMCP`, which is an **enum member** of an external catalogue package (`digital.vasic.llmsverifier/pkg/cliagents`), listed alongside `AgentGPTME`, `AgentOpenHands` etc. It names a known CLI agent type; it does not build, spawn, or import the vendored JavaScript.

Verdict: VENDORED-NOT-BUILT. These 139 advisories describe a dependency tree that is never installed and an artifact that cannot currently be produced. That is Ground A, and it is why they rank low — not because the CVEs are unimportant.

Corollary defect: a compose service pointing at a missing Dockerfile is itself broken and should be either fixed or removed. Filed as HXC-171.

`mcp-servers/postgres-mcp/ingest` — 13 advisories, evidence for Ground A

`mcp-servers/postgres-mcp` is a vendored copy of `@tigerdata/pg-aiguide`. Unlike GitMCP it **is** buildable and **is** in compose (`docker-compose.protocols.yml:224`). But all 13 advisories are against `ingest/uv.lock` — a Python docs-ingestion sub-tool — and its `Dockerfile` copies only:

```
COPY package.json bun.lock ./
COPY tsconfig.json ./
COPY src ./src
COPY skills ./skills
COPY skills.yaml ./
```

`ingest/` is **not copied into the image**. The Python dependencies (`langsmith`, `langchain-core`, `scrapy`, `Twisted`, `urllib3`, `cryptography`, `pyasn1`, `soupsieve`) are not present at runtime in the shipped container.

Verdict: NOT-IN-IMAGE. Ground A. Note this is a *narrower* claim than GitMCP's: the container around it does ship, so if `ingest/` is ever added to the Dockerfile, all 13 become live at once.

`plugins/mcp-server` — only 3 advisories, and the highest-priority npm cluster

This is the inversion the triage exists to surface. `plugins/mcp-server`:

- is **ours** — `"name": "@helixagent/mcp-server"`
- **has** a real `Dockerfile`
- **is** wired into `docker-compose.protocols.yml:22` as `helixagent-mcp-server`
- has `node_modules` present (it is actually installed and built)

Its 3 advisories carry **no Ground A and no Ground B**. By raw count it is 1.5% of the problem; by deployed surface it is the npm code most likely to be running.

Two reachability oracles, and what each can prove

Dependabot matches manifest version ranges. It proves "a vulnerable version is listed", never "we reach the vulnerable code".

`govulncheck` builds a call graph from our source to the vulnerable symbol. A *symbol-level* finding is strong evidence of reachability; absence of one is meaningful evidence of non-reachability (Ground B).

Two caveats recorded so the traces below are not over-read:

1. **Not all `govulncheck` traces are equally strong.** Many reported traces route through generic interface dispatch — e.g. *"*`handlers.UnifiedHandler.Completions` *calls* `client.httpError.Error`*"* or *"... calls* `io.Copy`*, which eventually calls* `client.hijackedConn.Read`*"*. These are artifacts of Go's method-set analysis: any `error.Error()` call site appears to reach every error type's `Error` method. Where a finding rests only on such traces, that is noted. Direct traces — e.g. *"*`transport.Start` *calls* `http3.Server.ListenAndServe`*"* — are strong.
2. **The two tools disagree, and Dependabot is not a superset.** `govulncheck` found a *reachable* vulnerability in `golang.org/x/text@v0.38.0` (**GO-2026-5970**, infinite loop on invalid input, fixed in `v0.39.0`) for which **there is no Dependabot alert at all**. Confirmed: no `golang.org/x/text` alert exists in the 204. A Dependabot-only view of Go risk is incomplete.

Ranked triage

Ranked by (reachability x severity), not severity alone.

Rank 1 — reachable, high severity, no fix exists: `github.com/docker/docker`

`go.mod` pins `v28.5.2+incompatible`. Four advisories, and `govulncheck` confirms we **call** the affected client surface directly. The call sites are real and specific — `internal/clis/openhands/sandbox.go` is a Docker-backed code-execution sandbox:

```
sandbox.go:158  Sandbox.Start     -> client.Client.ContainerCreate
sandbox.go:218  Sandbox.Execute   ->
client.Client.ContainerExecAttach
sandbox.go:312  Sandbox.CopyFrom  ->
client.Client.CopyFromContainer
sandbox.go:298  Sandbox.CopyTo    -> client.Client.CopyToContainer
sandbox.go:451  NewSandboxManager -> client.NewClientWithOpts
```

`CopyTo`/`CopyFrom` are exactly the `docker cp` API surface that three of the advisories concern:

GHSA	CVE	Sev	Subject	Fix
GHSA-rg2x-37c3-w2rh	CVE-2026-42306	high	race in `docker cp` -> bind-mount redirection to host path	**none**
GHSA-x86f-5xw2-fm2r	CVE-2026-41567	high	`PUT /containers/{id}/archive` executes container binary on host	**none**
GHSA-x744-4wpc-v9h2	CVE-2026-34040	high	AuthZ plugin bypass on oversized bodies	29.3.1
GHSA-vp62-88p7-qqf5	CVE-2026-41568	medium	race in `docker cp` -> arbitrary empty files on host	**none**
GHSA-pxq6-2prw-chj9	CVE-2026-33997	medium	off-by-one in plugin privilege validation	**none**

This is rank 1 because it is the only cluster that is **reachable, high-severity, and mostly unfixable by upgrading**. It needs a mitigation decision (is the OpenHands sandbox enabled in deployment? can `CopyTo`/`CopyFrom` be avoided or constrained? is the Docker socket exposed to untrusted input?), not a version bump. Filed as HXC-169.

Caveat (§11.4.6): `govulncheck` proves the *client* symbols are called. Several of these CVEs are daemon-side or require attacker influence over copy paths; **whether an attacker can influence those paths in our deployment was not determined here** and needs the dedicated analysis in HXC-169.

Rank 2 — reachable, high severity, fix available: `go-git` / `go-billy`

Called from two real call sites — `internal/checkpoints/checkpoint.go:57` (`git.PlainOpen`) and `internal/templates/resolver.go:202,205` (`git.Repository.Log`, commit iteration):

Module	Pinned	GHSA	Sev	Fix
`go-git/go-billy/v5`	v5.8.0	GHSA-qw64-3x98-g7q2 (CVE-2026-44973) path traversal	high	5.9.0
`go-git/go-billy/v5`	v5.8.0	GHSA-m3xc-h892-ggx6 (CVE-2026-44740) symlink cycle -> DoS	medium	5.9.0
`go-git/go-git/v5`	v5.18.0	GHSA-389r-gv7p-r3rp (CVE-2026-45022) object parsing divergence	high	5.19.0
`go-git/go-git/v5`	v5.18.0	GHSA-m7cr-m3pv-hgrp (CVE-2026-45570) SSH quote escaping	low	5.19.1

Reachable **and** fixable by a minor bump within v5. Highest value-per-risk in the whole set. Part of the cheap-wins cluster (HXC-170).

Rank 3 — reachable, no Dependabot alert: `golang.org/x/text`

GO-2026-5970, `v0.38.0` -> `v0.39.0`. Reachable via `internal/security/guardrails.go:679` (`SystemPromptProtector.Check` -> `digital.Normalize` -> `norm.Form.String`) — i.e. on a **security-guardrail input path that processes untrusted prompt text**, which is the worst place to have an infinite-loop-on-invalid-input defect. Also reached via `internal/services/debate_service.go:682`.

Ranked above several "high" advisories because it is reachable on an attacker-influenced path, trivially fixable, and **invisible to the Dependabot list** that the 204 figure comes from.

Rank 4 — reachable, fixable: `grpc`, `quic-go`, `otel`

Module	Pinned	Advisory	Sev	Fix	Trace strength
`google.golang.org/grpc`	v1.80.0	GHSA-hrxh-6v49-42gf / GO-2026-6061 xDS RBAC + HTTP/2	high	1.82.1	strong — `cmd/grpc-main.go:1204` `grpc.Server.Serve` `transport.NewServe`
`quic-go/quic-go`	v0.59.0	GHSA-vvgj-x9jq-8cj9 / GO-2026-5676	medium	0.59.1	strong — `internal/http3.go:124` `Server.ListenAndSe`

Module	Pinned	Advisory	Sev	Fix	Trace strength
		QPACK trailer memory exhaustion			`internal/router/quic_server.go:66 ListenAndServeTLS`
`otel .../otlptracehttp`	v1.40.0	GHSA-w8rr-5gcm-pp58 / GO-2026-4985 unbounded response bodies	medium	1.43.0	strong — `internal/observability/expo otlptracehttp.New`

`grpc` is a genuine server-side listener; `quic-go` backs a real HTTP/3 server. Both fixable by patch/minor bumps.

Rank 5 — ours and deployed, but reachability unknown: `plugins/mcp-server`

Package	Advisory	Sev	Fix
`@modelcontextprotocol/sdk`	GHSA-345p-7cg4-v4c7 — DNS rebinding protection not enabled by default	high	1.24.0 / 1.26.0
`ws`	memory-exhaustion DoS from tiny fragments	high	8.21.0

No Ground A (it ships) and no Ground B (no JS reachability analysis was run). The MCP SDK advisory is configuration-shaped — it concerns a default that must be turned on — which makes it worth checking directly rather than assuming the upgrade fixes it. Filed as HXC-172.

Rank 6 — `sdk/web` (22), and why it is not simply "dev-scope, ignore"

19 of 22 are `development` scope — `handlebars` (1 critical + 4 high), `js-yaml`, `flatted`, `minimatch`. These are build-toolchain packages: they do not execute in a user's runtime.

They are **not** dismissed, because `sdk/web` is `helixagent-sdk`, and `"private"` is **not** set — i.e. it is publishable. A code-injection defect in a *build* dependency of a package we publish is a supply-chain risk to

consumers of the SDK, which is a different question from runtime exploitability. Ground A does not cleanly apply. Left as undetermined-but-tracked rather than ranked low.

Rank 7 — T4/T5, Ground A only

GitMCP (139), postgres-mcp ingest (13), Website (6), research artifact (2), ci/reporter (1) — 161 advisories. Low priority **on Ground A only**: the affected trees are not built into anything we run (evidence above). No claim is made that these CVEs are harmless or unreachable in the abstract; if any of these directories is ever wired into a build, its advisories become live immediately and must be re-triaged.

The five critical advisories

Severity alone put these at the top; reachability does not keep them there. Four of five are **development**-scope, and three of five are in the unbuilt GitMCP tree.

#	Package	GHSA / CVE	Manifest	Scope	Fix	Assessm
1	`vitest`	GHSA-5xrq-8626-4rwp / CVE-2026-47429	`GitMCP/pnpm-lock.yaml`	development	3.2.6	Require the Vite **server be liste** — a develop running `vitest` Not shi GitMCP deps ne installe (Groun
2	`vitest`	same, second manifest	`GitMCP/package.json`	development	3.2.6	Duplica #1 acro two manifes the sam unbuilt
3	`handlebars`	GHSA-2w6w-674q-4c4q / CVE-2026-33937	`sdk/web/package-lock.json`	development	4.7.9	JS injec via AST confusi

#	Package	GHSA / CVE	Manifest	Scope	Fix	Assessm
						Build-toolcha a **publish** SDK -> supply-chain relevan (see Ra 6). Transiti no dire pin.
4	`@react-router/node`	GHSA-9583-h5hc-x8cw / CVE-2025-61686	`GitMCP/pnpm-lock.yaml`	**runtime**	7.9.4	Path traversa file sess storage **only runtim scope critical** Belongs the vendor Cloudfl Worker app we not bui (Groun
5	`form-data`	GHSA-fjxv-7rqg-78g4 / CVE-2025-7783	`GitMCP/pnpm-lock.yaml`	development	4.0.4	Unsafe random multipa bounda Unbuilt (Groun

Net: zero of the five criticals is currently reachable in a deployed artifact, each on Ground A or on documented scope — and none of that is a reason to leave them permanently unpatched, because the moment GitMCP is wired into a build, #1, #2, #4 and #5 go live together.

Go: proven-reachable set

`govulncheck` symbol-level: **11 vulnerabilities across 7 modules are called by our code.**

#	GO id	Module	Pinned	Fix	Sev (GHSA)	Strongest trace
1	GO-2026-6061	`grpc`	1.80.0	1.82.1	high	`cmd/grpc-server/main.go:1204 -> Server.Serve`
2	GO-2026-5970	`x/text`	0.38.0	0.39.0	*no alert*	`internal/security/guardrails.go:679`
3	GO-2026-5676	`quic-go`	0.59.0	0.59.1	medium	`internal/transport/http3.go:124`
4	GO-2026-5668	`docker`	28.5.2	none	medium	`sandbox.go:298 CopyToContainer`
5	GO-2026-5597	`go-billy`	5.8.0	5.9.0	high	`checkpoint.go:57 git.PlainOpen`
6	GO-2026-5496	`go-git`	5.18.0	5.19.1	low	`resolver.go:202 Repository.Log`
7	GO-2026-5490	`go-billy`	5.8.0	5.9.0	medium	`checkpoint.go:57`
8	GO-2026-5074	`go-git`	5.18.0	5.19.0	high	`resolver.go:202/205`
9	GO-2026-4985	`otel otlptracehttp`	1.40.0	1.43.0	medium	`exporter.go:88`
10	GO-2026-4887	`docker`	28.5.2	none	high	`sandbox.go:158/218`
11	GO-2026-4883	`docker`	28.5.2	none	medium	`sandbox.go:158/218`

Go: proven-NOT-reachable set

`govulncheck -show verbose`: 3 imported-but-not-called, 4 required-but-not-imported. This is Ground B — the strongest available basis for deprioritising a Go advisory.

Level	GO id	Module	Pinned	Fix
imported, not called	GO-2026-5841	`klauspost/compress`	1.18.5	1.18.7
imported, not called	GO-2026-5693	`go-git/v5`	5.18.0	5.19.1

Level	GO id	Module	Pinned	Fix
imported, not called	GO-2026-5336	`go-git/v5`	5.18.0	5.19.1
required, not imported	GO-2026-5932	`x/crypto`	0.53.0	none
required, not imported	GO-2026-5746	`docker`	28.5.2	none
required, not imported	GO-2026-5617	`docker`	28.5.2	none
required, not imported	GO-2026-4945	`go-jose/v3`	3.0.4	3.0.5

Of note: **`go-jose/v3` GHSA-78h2-9frx-2jm8 (high, JWE decryption panics) is required-but-not-imported** — a high-severity alert on the Dependabot list that Ground B places at the bottom. This is a case where the two oracles differ and the reachability answer is the more useful one.

Cheap wins

Advisories fixed by a patch/minor bump within the same major version — no API break expected — grouped so they can land as a small number of reviewable changes. **None of these were applied in this pass.**

Cluster A — Go, reachable, patch/minor only (the recommended first change, HXC-170). One `go get` batch, no major versions crossed:

Module	From	To	Closes
`github.com/go-git/go-git/v5`	5.18.0	5.19.1	4 alerts (1 high, 1 low, 2 med) + 2 unreachable
`github.com/go-git/go-billy/v5`	5.8.0	5.9.0	2 alerts (1 high, 1 med)
`golang.org/x/text`	0.38.0	0.39.0	GO-2026-5970 (reachable, no alert)
`github.com/quic-go/quic-go`	0.59.0	0.59.1	1 med (reachable)
`google.golang.org/grpc`	1.80.0	1.82.1	1 high (reachable) x3 manifests
	1.40.0	1.43.0	1 med (reachable)

Module	From	To	Closes
`go.opentelemetry.io/otel/exporters/.../otlptracehttp`			
`github.com/klauspost/compress`	1.18.5	1.18.7	1 (unreachable)
`github.com/go-jose/go-jose/v3`	3.0.4	3.0.5	1 high (unreachable)

Covers **all 11 reachable Go vulnerabilities except the 3 unfixable Docker ones**, plus the entire unreachable Go set bar Docker/x-crypto.

Caveat: `grpc` 1.80.0 -> 1.82.1 is a **minor** bump, not patch. It should still be low-risk but is the one entry in Cluster A warranting a compile-and-test check. `x/text` 0.38 -> 0.39 is a golang.org/x pre-1.0 minor — conventionally backward-compatible, but likewise worth the check.

Cluster B — `plugins/mcp-server` (ours, deployed; HXC-172): `ws` -> 8.21.0 (patch/minor), `@modelcontextprotocol/sdk` -> 1.24.0+. Small lockfile, 3 alerts, genuinely shipped — best effort-to-exposure ratio on the npm side.

Not cheap wins, despite looking like it: the 139 GitMCP advisories. A lockfile refresh there would be a large diff against a vendored upstream tree we do not build — high review cost, zero change to deployed exposure. The correct action is the vendoring decision (HXC-171), not a bump.

No fix available

6 of 204 have **no patched version** — they cannot be closed by upgrading:

Sev	Package	GHSA	Location
high	`github.com/docker/docker`	GHSA-rg2x-37c3-w2rh	`go.mod` — **reachable**
high	`github.com/docker/docker`	GHSA-x86f-5xw2-fm2r	`go.mod` — **reachable**
medium	`github.com/docker/docker`	GHSA-vp62-88p7-qqf5	`go.mod` — **reachable**
medium	`github.com/docker/docker`	GHSA-pxq6-2prw-chj9	`go.mod` — **reachable**

Sev	Package	GHSA	Location
high	`scrapy`	GHSA-h7wm-ph43-c39p	postgres-mcp `ingest/` — not in image
low	`@ai-sdk/provider-utils`	GHSA-866g-f22w-33x8	GitMCP — not built

Plus `x/crypto` GO-2026-5932 (no fix, not imported).

The four Docker entries are the residue that no upgrade campaign can clear, and they are the ones we actually reach. That is the substance of HXC-169.

Transitive-only advisories

Where we do not control the version directly (`relationship: transitive`), a fix requires the direct parent to move, or a lockfile override:

- **51 runtime-transitive + 43 development-transitive = 94 of 204.**
- Go transitive-and-reachable: `go-billy/v5` (both advisories — pulled by `go-git`, so the Cluster A `go-git` bump should carry it; **verify the resolved version after bumping rather than assuming**), `go-jose/v3` (unreachable).
- npm criticals #3 (`handlebars`) and #5 (`form-data`) are transitive; #4 (`@react-router/node`) is transitive.
- 16 alerts have `relationship: unknown` — including the `grpc` high across three manifests — so even the direct/transitive split is not fully known.

What could NOT be determined

Stated plainly, because these are the gaps that matter more than the parts that resolved cleanly.

1. **No reachability analysis was performed for 184 of 204 advisories** (171 npm + 13 pip). `govulncheck` is Go-only. The npm/pip advisories were classified by **deployment surface (Ground A) and declared scope**, never by call-path. Specifically:
    - The 139 GitMCP advisories are Ground A (not built). **Not** assessed for reachability. If the Dockerfile is ever added, we have no reachability data.

    - The 22 `sdk/web` advisories: **19 dev-scope, 3 runtime** — the 3 runtime ones were not reachability-assessed at all.
    - The 3 `plugins/mcp-server` advisories — our genuinely shipped code — were **not** reachability-assessed. This is the most consequential gap, because it is the one place with neither Ground A nor Ground B.
    - Tools that would close this (`npm audit --omit=dev` against an installed tree, `osv-scanner`, commercial reachability scanners) were **not** run.

2. **Exploitability of the reachable Docker advisories was not established.** `govulncheck` proves we call `CopyToContainer`/`CopyFromContainer`. Whether an attacker can influence the paths or archive contents those calls handle depends on how the OpenHands sandbox is exposed, and that analysis was not done.

3. **Whether the OpenHands Docker sandbox is enabled in any real deployment** was not determined. If it is compiled in but never enabled by configuration, the rank-1 cluster's practical severity drops considerably — but that is a configuration question this pass did not answer, and it is not safe to assume either way.

4. **`govulncheck` analysed the default build.** Code behind build tags (e.g. `nogui`) or platform-specific files may add call paths not represented here.

5. **Runtime/dev scope is Dependabot's classification, taken on trust.** It was not independently verified against the installed trees.

6. **Dependabot is enabled on one mirror only** (`vasic-digital`, not `HelixDevelopment`). Whether other owned repos have similarly unreported advisory backlogs was not checked.

7. **`govulncheck` found one reachable vulnerability Dependabot did not report** (`x/text`). This proves the 204 figure is not an upper bound on Go risk. **No equivalent cross-check was possible for npm/pip**, so the true npm/pip count may also exceed what Dependabot reports.

Honest boundaries

- **This triage does not reduce our exposure by one advisory.** Nothing was upgraded. 204 remain open.

- **The component is not free of known vulnerabilities.** 11 Go vulnerabilities are proven-called today, 4 of them with no available fix.
- **"Not deployed" is a statement about the current tree**, verified at `c740eaf1`. It is invalidated by anyone adding a Dockerfile, a workspace glob, or a COPY line. The GitMCP and postgres-mcp-ingest conclusions in particular are one commit away from being wrong, which is why they are filed as decisions (HXC-171) rather than dismissals.
- **Absence of a `govulncheck` finding is good evidence, not proof.** Reflection, `go:linkname`, cgo, and dynamic dispatch can defeat static call-graph analysis.

Recommended follow-up items

Item	Type	Sev	Subject
HXC-169	Bug	Critical	Reachable, unfixable Docker `cp` advisories — mitigation decision
HXC-170	Task	High	Cheap-wins cluster: 8 Go modules, patch/minor, closes 11 of 18 Go advisories
HXC-171	Bug	Medium	GitMCP vendoring decision + broken `mcp-gitmcp` compose service
HXC-172	Task	High	`plugins/mcp-server` — reachability-assess and patch the 3 shipped npm advisories

Sources verified 2026-07-29

- GitHub Dependabot Alerts REST API — `GET /repos/{owner}/{repo}/dependabot/alerts`, API version `2022-11-28` (response header `X-Github-Api-Version-Selected`).
- `govulncheck` from the Go vulnerability database (`https://pkg.go.dev/vuln/`); per-advisory detail pages `https://pkg.go.dev/vuln/GO-2026-XXXX`.
- Advisory metadata (CVE ids, CWE ids, CVSS vectors, patched versions) as returned inline by the Dependabot API payload.
- Local build evidence read directly from the tree at `c740eaf1`: `docker-compose.protocols.yml`, `mcp-servers/postgres-mcp/Dockerfile`, `mcp-servers/GitMCP/package.json`, `cmd/helixagent/main.go`.